Mansi Davkhar

D20C / 13

Blockchain Experiment No. 10

Aim

To implement a Hyperledger Fabric network, deploy chaincode, and perform asset creation, querying, and transfer operations.

Theory

Hyperledger Fabric is a permissioned, modular blockchain platform designed for enterprise applications. Unlike public blockchains, participants in a Fabric network have known identities managed by organizations. A network consists of peers, an ordering service, organizations, and channels. Peers maintain the ledger and execute chaincode, while the ordering service establishes the order of transactions.

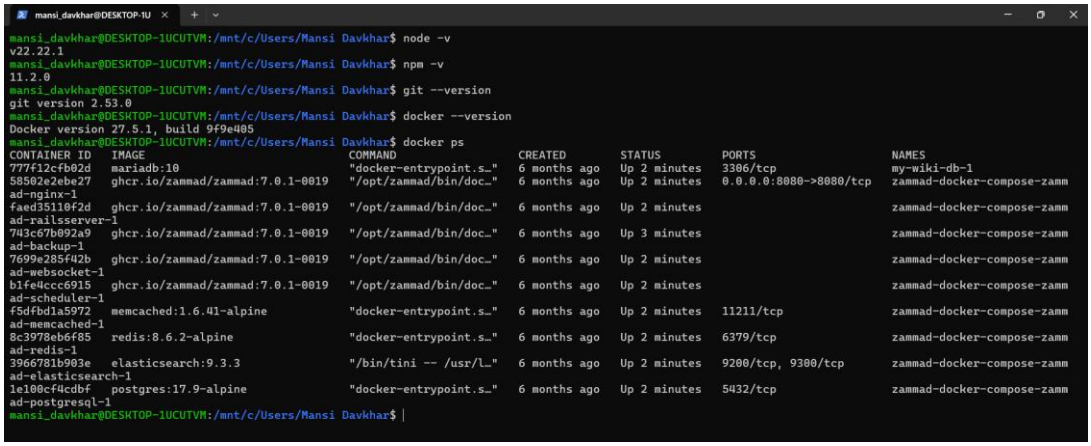
Chaincode is the smart-contract logic that defines the operations permitted on ledger data. In this experiment, the JavaScript asset-transfer chaincode is deployed on a sample Fabric network. The ledger is initialized with sample assets, queried to retrieve records, and updated by creating a new asset and transferring its ownership. Query operations read ledger state, while invoke operations submit transactions that can update the ledger.

Procedure

Step 1: Verify prerequisites

Open Ubuntu in WSL 2 and verify that Docker, Git, Node.js, and npm are available. Docker Desktop must be running and WSL integration enabled.

docker --version
docker ps
git --version
node -v
npm -v



The commands confirmed the installed tools and verified that Docker could communicate with its daemon.

Step 2: Obtain Fabric Samples and install binaries

The official Hyperledger Fabric samples repository was cloned, and the installation script was used to download the Fabric CLI binaries and Docker images.

```
git clone https://github.com/hyperledger/fabric-samples
cd fabric-samples
curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
```

```
mansi_davkhar@DESKTOP-1UCUTVM:/mnt/c/Users/Mansi Davkhar$ git clone https://github.com/hyperledger/fabric-samples
Cloning into 'fabric-samples'...
remote: Enumerating objects: 16063, done.
remote: Counting objects: 100% (3470/3470), done.
remote: Compressing objects: 100% (641/641), done.
remote: Total 16063 (delta 3045), reused 2829 (delta 2829), pack-reused 12593 (from 3)
Receiving objects: 100% (16063/16063), 24.08 MiB | 1.88 MiB/s, done.
Resolving deltas: 100% (9276/9276), done.
Updating files: 100% (1128/1128), done.
mansi_davkhar@DESKTOP-1UCUTVM:/mnt/c/Users/Mansi Davkhar$ cd fabric-samples
mansi_davkhar@DESKTOP-1UCUTVM:/mnt/c/Users/Mansi Davkhar/fabric-samples$
```

Figure 1: Cloning the Hyperledger Fabric samples repository.

```
mansi_davkhar@DESKTOP-1UCUTVM:/mnt/c/Users/Mansi Davkhar/fabric-samples$ docker images | grep fabric
hyperledger/fabric-peer              2.5               a3835b028353   4 months ago    155MB
hyperledger/fabric-peer              2.5.16            a3835b028353   4 months ago    155MB
hyperledger/fabric-peer              latest            a3835b028353   4 months ago    155MB
ghcr.io/hyperledger/fabric-peer      2.5.16            a3835b028353   4 months ago    155MB
hyperledger/fabric-orderer           2.5               de354d43b52b   4 months ago    121MB
hyperledger/fabric-orderer           2.5.16            de354d43b52b   4 months ago    121MB
hyperledger/fabric-orderer           latest            de354d43b52b   4 months ago    121MB
ghcr.io/hyperledger/fabric-orderer   2.5.16            de354d43b52b   4 months ago    121MB
hyperledger/fabric-ccenv             2.5               c011802bedda   4 months ago    675MB
hyperledger/fabric-ccenv             2.5.16            c011802bedda   4 months ago    675MB
hyperledger/fabric-ccenv             latest            c011802bedda   4 months ago    675MB
ghcr.io/hyperledger/fabric-ccenv     2.5.16            c011802bedda   4 months ago    675MB
hyperledger/fabric-baseos            2.5               94ad0ab8a77f   4 months ago    160MB
hyperledger/fabric-baseos            2.5.16            94ad0ab8a77f   4 months ago    160MB
hyperledger/fabric-baseos            latest            94ad0ab8a77f   4 months ago    160MB
ghcr.io/hyperledger/fabric-baseos    2.5.16            94ad0ab8a77f   4 months ago    160MB
ghcr.io/hyperledger/fabric-ca        1.5.17            0548296e36d4   8 months ago    244MB
hyperledger/fabric-ca                1.5               0548296e36d4   8 months ago    244MB
hyperledger/fabric-ca                1.5.17            0548296e36d4   8 months ago    244MB
hyperledger/fabric-ca                latest            0548296e36d4   8 months ago    244MB
mansi_davkhar@DESKTOP-1UCUTVM:/mnt/c/Users/Mansi Davkhar/fabric-samples$
```

Figure 2: Fabric Docker images downloaded and verified.

Step 3: Configure the working directory

The project was moved to the Ubuntu Linux filesystem under ~/blockchain-lab/fabric-samples to avoid path-handling problems caused by the Windows-mounted directory containing a space in its name. The test network commands were then run from the test-network directory.

```
cd ~/blockchain-lab/fabric-samples/test-network
pwd
```

Step 4: Start the network and create a channel

The test network was started and the channel mychannel was created. The network includes two peer organizations and an ordering service. Each peer joins the channel and maintains ledger data.

```
./network.sh up createChannel
```

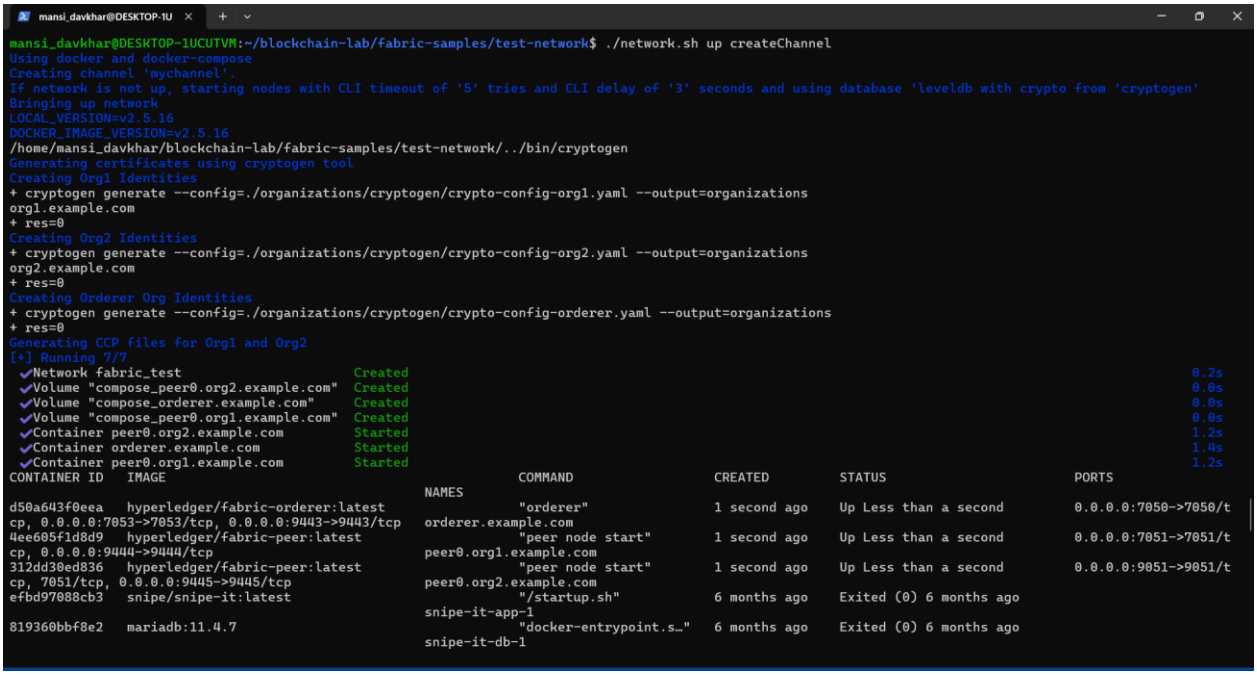


Figure 3: Fabric test network and channel setup.

Step 5: Deploy chaincode

The JavaScript asset-transfer chaincode was deployed with the name basic.

```
./network.sh deployCC \
  -ccn basic \
  -ccp ../asset-transfer-basic/chaincode-javascript \
  -ccl javascript
```

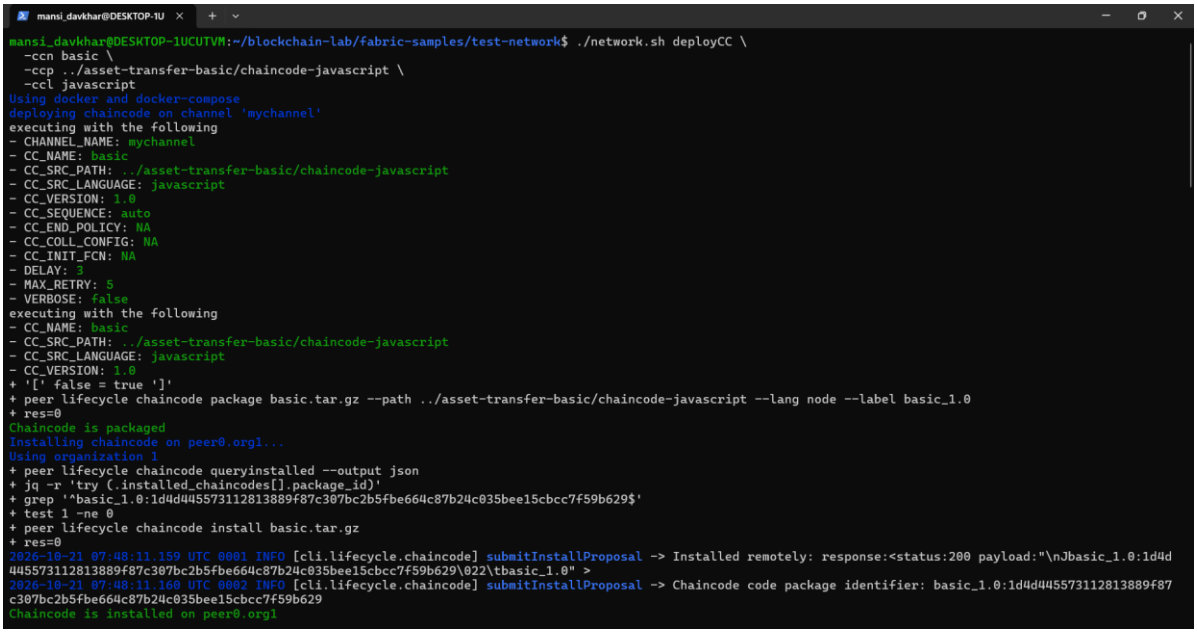


Figure 4: Chaincode deployment and network command output.

Step 6: Set the peer environment

Environment variables were configured so that the Fabric CLI could connect to Org1's peer using its administrator identity and TLS certificate.

```
export PATH="${PWD}/../bin:$PATH"
export FABRIC_CFG_PATH="${PWD}/../config"
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export
CORE_PEER_TLS_ROOTCERT_FILE="${PWD}/organizations/peerOrganizations/org1.example.com/peer
s/peer0.org1.example.com/tls/ca.crt"
export
CORE_PEER_MSPCONFIGPATH="${PWD}/organizations/peerOrganizations/org1.example.com/users/Ad
min@org1.example.com/msp"
export CORE_PEER_ADDRESS="localhost:7051"
export
ORDERER_CA="${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/
msp/tlscacerts/tlsca.example.com-cert.pem"
```

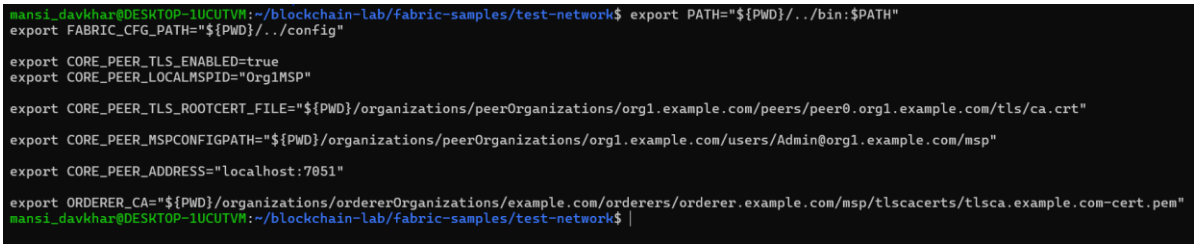


Figure 5: Peer environment configuration.

Step 7: Initialize the ledger

The InitLedger function was invoked to add the sample assets to the ledger.

```
peer chaincode invoke -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com --tls \
  --cafile "$ORDERER_CA" -C mychannel -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles
"${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt" \
  -c '{"function":"InitLedger","Args":[]}'
```

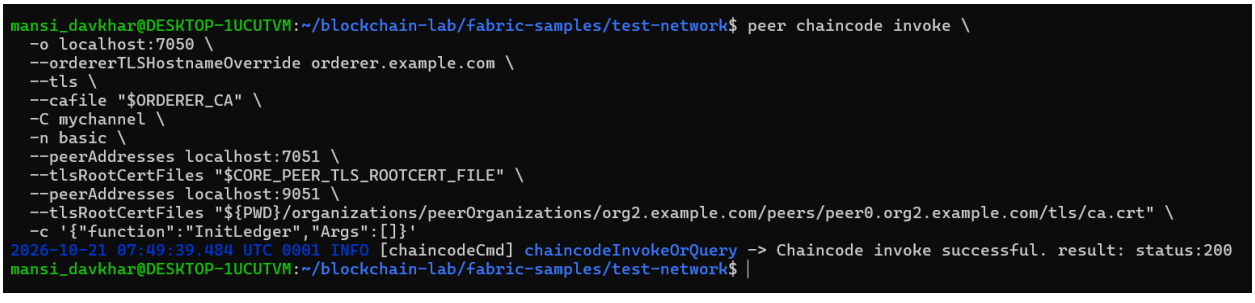


Figure 6: Ledger initialization and transaction output.

Step 8: Query all assets and read a specific asset

GetAllAssets was used to retrieve all assets. ReadAsset was then used to read the record for asset5. These are read-only queries and do not modify the ledger.

peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'

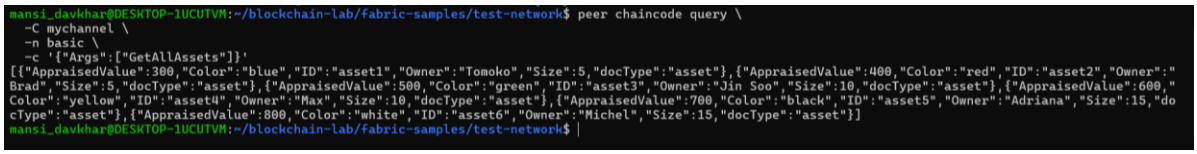


Figure 7: Querying assets from the ledger.

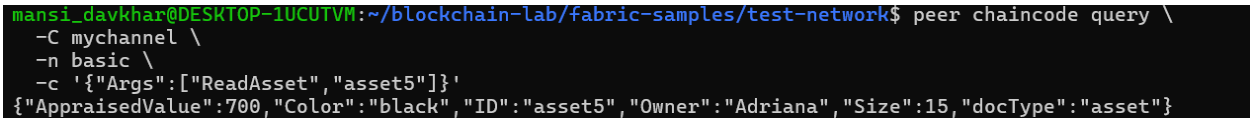


Figure 8: Reading an individual asset.

Step 9: Create a new asset

A new asset with ID asset13 was created with the attributes blue, size 5, owner Mansi, and appraised value 300.

peer chaincode invoke -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com --tls \
--cafile "$ORDERER_CA" -C mychannel -n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles
"${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt" \
-c '{"function":"CreateAsset","Args":["asset13","blue","5","Mansi","300"]}'



Figure 9: Asset creation and verification output.

Step 10: Transfer asset ownership

The ownership of asset11 was transferred to Alice using TransferAsset. The asset was queried again to verify that the owner field had changed.

peer chaincode invoke -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com --tls \
--cafile "$ORDERER_CA" -C mychannel -n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles
"${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt" \

-c '{"function":"TransferAsset","Args":["asset13","Yash"]}'

peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset13"]}'

```
mansi_davkhar@DESKTOP-1UCUTVM:~/blockchain-lab/fabric-samples/test-network$ peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "$ORDERER_CA" \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles "${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt" \
  -c '{"function":"TransferAsset","Args":["asset13","Yash"]}'
2026-10-22 08:10:18.901 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"Mansi"
```

Figure 10: Asset transfer and updated ownership.

```
mansi_davkhar@DESKTOP-1UCUTVM:~/blockchain-lab/fabric-samples/test-network$ peer chaincode query \
  -C mychannel \
  -n basic \
  -c '{"Args":["ReadAsset","asset13"]}'
{"AppraisedValue":300,"Color":"blue","ID":"asset13","Owner":"Yash","Size":5}
```

Figure 11: Final asset query showing the updated ledger state.

Step 11: Network Shutdown

```
mansi_davkhar@DESKTOP-1UCUTVM:~/blockchain-lab/fabric-samples/test-network$ ./network.sh down
Using docker and docker-compose
Stopping network
[+] Running 11/11
 ✔Container peer0.org1.example.com       Removed                                                                    3.0s
 ✔Container peer0.org2.example.com       Removed                                                                    3.1s
 ✔Container orderer.example.com          Removed                                                                    2.2s
 ✔Volume compose_orderer3.example.com    Removed                                                                    0.1s
 ✔Volume compose_peer0.org3.example.com  Removed                                                                    0.1s
 ✔Volume compose_orderer4.example.com    Removed                                                                    0.0s
 ✔Volume compose_peer0.org2.example.com  Removed                                                                    0.1s
 ✔Volume compose_orderer2.example.com    Removed                                                                    0.1s
 ✔Volume compose_peer0.org1.example.com  Removed                                                                    0.1s
 ✔Volume compose_orderer.example.com     Removed                                                                    0.2s
 ✔Network fabric_test                    Removed                                                                    0.5s
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59b629-c68f8ecc2a0ff0cc1497615f5e33a8bfa908e0fd3555f864cfa1398b52d341cb:latest
Deleted: sha256:934b46a39a7e76ca1edd09941823574 83e7661defd8f118ece2531a245abd9fe
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59b629-d3cf9b1f4b9747cc8433ac0e74bfaeade1cf0b97f5c1163835f604aa885d60dc:latest
Deleted: sha256:dbe62eabbcad402e11d33723decb05a266a8ae2f77d24a3e7728b825d9ad5f60
mansi_davkhar@DESKTOP-1UCUTVM:~/blockchain-lab/fabric-samples/test-network$
```

Inference

The experiment demonstrated that a Hyperledger Fabric network can be configured with peer organizations and a channel, and that JavaScript chaincode can be deployed to manage ledger assets. Query operations retrieved asset information, while invoke operations created an asset and changed its owner. The final query confirmed the ownership update.

Conclusion

Hyperledger Fabric was used to implement a permissioned blockchain network and perform asset management through chaincode. The experiment illustrated the role of peers, channels, TLS-based communication, and smart contracts in maintaining and updating a shared ledger.